

### 1) A well-defined base case exists.

The base case is  $\text{left} \geq \text{right}$ , which returns true. Intuitively, when the active window has length 0 (empty vector or all interior elements consumed) or 1 (a single middle element), the sequence is trivially a palindrome. This base case is both **necessary** (we must eventually stop) and **sufficient** (no further checks are needed once indices cross or meet).

### 2) Each recursive call makes progress toward the base case.

Progress is guaranteed by shrinking the interval from both ends: the call `isPalindrome(vec, left + 1, right - 1)` strictly reduces the problem size by 2. Since `left` increases monotonically and `right` decreases monotonically, the measure `right - left` is strictly reduced on every successful comparison. As vectors are finite, after at most  $\lceil n/2 \rceil$  steps the measure becomes  $\leq 0$ , triggering the base case. Additionally, the algorithm performs **useful work before recursing** by checking `vec[left] != vec[right]`. A mismatch immediately returns false, preventing unnecessary recursion and ensuring correctness.

### 3) Parameters for the recursive call are constructed deterministically.

Given inputs (`vec`, `left`, `right`), the next parameters are **uniquely determined**:

- If  $\text{left} \geq \text{right}$ : return (no recursive call).
- Else if `vec[left] != vec[right]`: return false (no recursive call).
- Else: recurse with (`vec`, `left + 1`, `right - 1`).

There is no branching that introduces ambiguity about the subproblem to solve next; when a recursive call occurs, its arguments are computed by a fixed rule (increment/decrement by 1). Thus, for the same input vector the function always follows the same call sequence and yields the same result—satisfying determinism.